Rapport – Mini-projet n°2

Du poste Windows au web sans mot de passe : Kerberos (SPNEGO), Keycloak et SAML 2.0

Nom : Aziz Chalbi
Établissement : Polytechnique Sousse, 3ème année cybersécurité
Matière : Gestion des Accès-Identités
Enseignant : Dr. Salah Gontara
Année : 2026-27

Résumé

Un employé du domaine polycorp.test ouvre sa session Windows sur CL01, lance Edge et ouvre l'application web de l'entreprise. L'application s'ouvre directement, sans aucune saisie. Son ticket Kerberos, obtenu à l'ouverture de session, est présenté au fournisseur d'identité Keycloak par HTTP Negotiate (SPNEGO). Keycloak le vérifie avec un keytab, retrouve l'utilisateur et ses groupes dans Active Directory par LDAP, puis émet une assertion SAML signée que l'application accepte.

J'ai monté cette chaîne de bout en bout sur Azure, en reprenant les VM du TP2 (DC01 et CL01) et en ajoutant une VM Linux sso.polycorp.test pour Keycloak et l'application. Toutes les démonstrations demandées sont faites avec des traces réelles : klist, événements 4769 / 4768 / 4771 sur DC01, journaux et événements Keycloak, capture réseau du dialogue Negotiate et contenu de l'assertion SAML. Le code, les scripts et la configuration sont dans le dépôt GitHub du projet (dossier projet-sso-kerberos-keycloak).

1. Objectif et cahier des charges

Le sujet demande quatre réalisations et quatre démonstrations. Voici où chacune se trouve dans ce rapport :

Exigence du sujet	Réalisation	Section
Keycloak joignable depuis CL01 par un nom DNS du domaine	Enregistrements A sso et app → 192.168.50.30 dans la zone polycorp.test	3.1
Compte de service, SPN et keytab (setspn, ktpass)	svc-kc-http, SPN HTTP/sso.polycorp.test, keytab AES-256 kvno 3	3.2
AD dans Keycloak (LDAP), intégration Kerberos, étape Kerberos du flux	Fédération ad-polycorp READ_ONLY, Kerberos activé, flux browser Kerberos = ALTERNATIVE	3.4
Site autorisé pour l'authentification intégrée, par GPO	GPO PRJ-SSO-Navigateur liée à Workstations	3.3
Application ouverte sans saisie ; klist montre le ticket HTTP	Session de Bob, ticket HTTP/sso.polycorp.test	4.1
4769 sur DC01 ; assertion SAML avec les attributs AD	4769 Service Name svc-kc-http ; email, upn, title, memberOf	4.2, 4.3
Hors domaine ou sans ticket : retour au formulaire	Poste Linux hors domaine et session locale sur CL01	5.1
Compte désactivé, décalage d'horloge : refus expliqués avec trace	4768 0x12 / Kerberos (18) ; Clock skew too great (37)	5.2, 5.3

2. Architecture

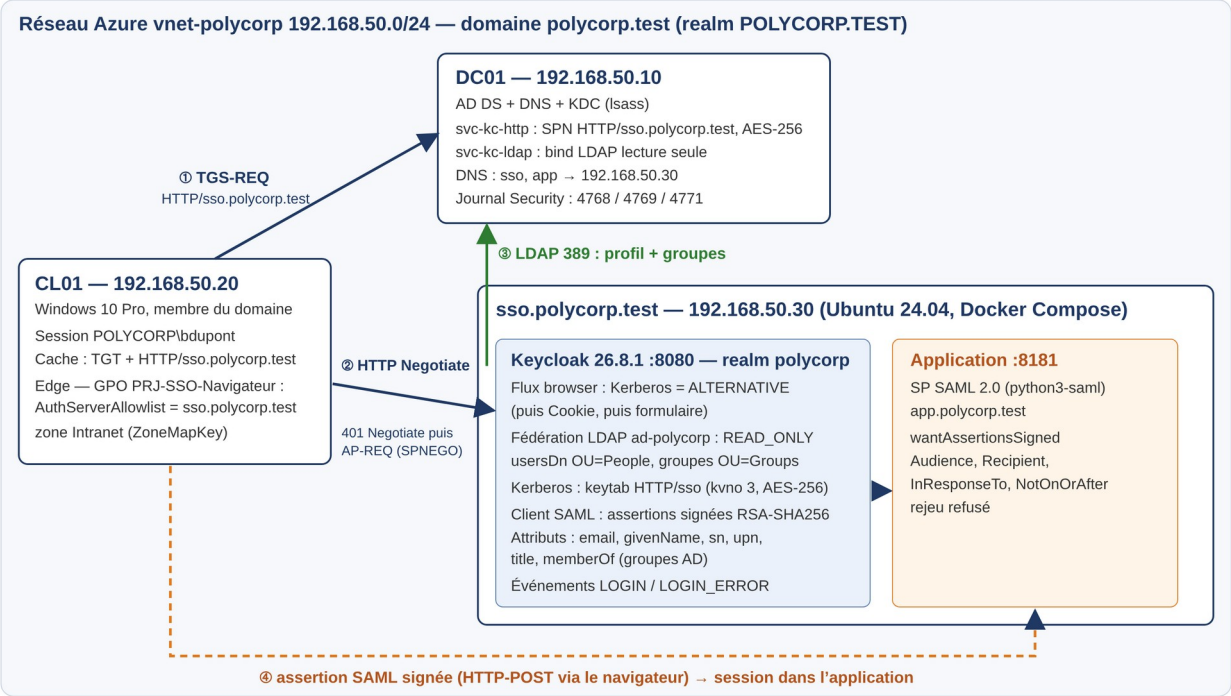


Figure 1 – Architecture du projet : CL01, DC01 et la VM sso.polycorp.test (Keycloak + application).

Les trois machines sont dans le même réseau virtuel Azure, 192.168.50.0/24, dont le DNS est DC01. Seul mon poste peut joindre les VM depuis Internet (NSG : RDP, SSH et les ports 8080/8181 depuis mon adresse IP uniquement).

Machine	Adresse	Système	Rôle
DC01	192.168.50.10	Windows Server 2019	AD DS, DNS, KDC du realm POLYCORP.TEST
CL01	192.168.50.20	Windows 10 Pro 22H2	Poste du domaine, navigateur Edge
sso	192.168.50.30	Ubuntu 24.04, Docker	Keycloak 26.8.1 (IdP) et application SP SAML

Le déroulement d'un accès, étape par étape :

- 1. Bob ouvre http://app.polycorp.test:8181/rh. L'application n'a pas de session et redirige le navigateur vers Keycloak avec une AuthnRequest SAML (binding Redirect).
- 2. Keycloak exécute le flux browser. La première étape est Kerberos : il répond 401 avec WWW-Authenticate: Negotiate.
- 3. Edge voit que sso.polycorp.test est autorisé par la GPO. Il demande au KDC un ticket de service pour HTTP/sso.polycorp.test (TGS-REQ, événement 4769) et renvoie la requête avec Authorization: Negotiate, qui contient un jeton SPNEGO avec l'AP-REQ Kerberos.
- 4. Keycloak déchiffre le ticket avec le keytab : c'est la preuve que le KDC a authentifié bdupont@POLYCORP.TEST. Il retrouve bdupont dans AD par LDAP (profil et groupes), ouvre sa session, puis renvoie au navigateur une assertion SAML signée (binding POST).

– 5. L'application vérifie la signature, l'Audience, le Recipient, InResponseTo et la validité, puis crée sa session locale.

Le mot de passe de Bob n'est envoyé ni à Keycloak ni à l'application. Il n'a été utilisé qu'une fois, à l'ouverture de session Windows.

3. Réalisation

3.1 La VM sso et le DNS

J'ai créé la VM PRJ-SSO avec Azure CLI dans le même groupe de ressources que le TP2 (Ubuntu 24.04, Standard_B2als_v2, IP fixe 192.168.50.30, accès SSH par clé seulement), puis installé Docker et les outils krb5. Comme le réseau virtuel utilise DC01 comme DNS, la VM résout directement les noms du domaine (getent hosts dc01.polycorp.test donne 192.168.50.10). Sur DC01, j'ai ajouté deux enregistrements A dans la zone polycorp.test :

```
HostName IP
-------- --
app      192.168.50.30
CL01     192.168.50.20
dc01     192.168.50.10
sso      192.168.50.30
```

J'ai utilisé deux noms pour la même machine : sso pour l'IdP et app pour l'application. C'est plus proche d'un vrai déploiement, et ça montre que seul le nom de l'IdP a besoin d'un SPN et d'une autorisation dans le navigateur.

3.2 Comptes de service, SPN et keytab

J'ai créé une OU Services avec deux comptes séparés, chacun avec un seul rôle :

– svc-kc-ldap : utilisé par Keycloak pour le bind LDAP. Il est simple membre de Domain Users, donc en lecture seule sur l'annuaire.

– svc-kc-http : porte le SPN HTTP/sso.polycorp.test. Sa clé chiffre les tickets que les postes présentent à Keycloak.

Les deux comptes ont un mot de passe aléatoire long, n'expirent pas (comme un compte de service classique) et sont marqués « Account is sensitive and cannot be delegated ». Pour svc-kc-http, j'ai limité les types de chiffrement à AES (msDS-SupportedEncryptionTypes = 24), pour que le KDC ne délivre jamais de ticket RC4 pour ce service.

```
setspn -S HTTP/sso.polycorp.test POLYCORP\svc-kc-http
Registered ServicePrincipalNames for CN=svc-kc-http,OU=Services,DC=polycorp,DC=test:
        HTTP/sso.polycorp.test
ktpass -out C:\Users\Public\sso.keytab -princ HTTP/sso.polycorp.test@POLYCORP.TEST -mapuser
POLYCORP\svc-kc-http -pass ******** -crypto AES256-SHA1 -ptype KRB5_NT_PRINCIPAL
Successfully mapped HTTP/sso.polycorp.test to svc-kc-http.
keysize 87 HTTP/sso.polycorp.test@POLYCORP.TEST ptype 1 (KRB5_NT_PRINCIPAL) vno 3 etype 0x12 (AES256-
SHA1)
```

setspn -S vérifie d'abord qu'aucun autre compte ne porte déjà ce SPN. Un SPN en double empêcherait le KDC de choisir la bonne clé et l'authentification échouerait. ktpass recalcule la clé à partir du mot de passe, avec le sel du nouveau principal, et l'écrit dans le keytab. Le KVNO est 3 parce que ktpass a changé le mot de passe du compte. J'ai vérifié le keytab sur la VM :

```
klist -k -e -t sso.keytab
KVNO Principal
---- --------------------------------------------------
   3 HTTP/sso.polycorp.test@POLYCORP.TEST (aes256-cts-hmac-sha1-96)
```

Le keytab est monté en lecture seule dans le conteneur Keycloak (chmod 600 sur l'hôte), et il n'est jamais mis dans le dépôt Git. Avec lui, on pourrait se faire passer pour le service, il faut donc le protéger comme un mot de passe.

3.3 GPO navigateur

Par défaut, un navigateur n'envoie pas de ticket Kerberos à n'importe quel site : il faut lui dire à qui il peut faire confiance. J'ai créé la GPO ordinateur PRJ-SSO-Navigateur, liée à l'OU Workstations (là où se trouve CL01) :

```
HKLM\SOFTWARE\Policies\Microsoft\Edge
   AuthServerAllowlist     = sso.polycorp.test
   HideFirstRunExperience  = 1
HKLM\Software\Policies\Microsoft\Windows\CurrentVersion\Internet Settings\ZoneMapKey
   http://sso.polycorp.test = 1   (zone Intranet local)
```

AuthServerAllowlist est le réglage d'Edge pour l'authentification intégrée. La Site to Zone Assignment List met aussi le site dans la zone Intranet pour Windows (utilisée par exemple par PowerShell). Après gpupdate /target:computer /force, CL01 avait bien les deux valeurs. Je n'ai mis que sso.polycorp.test, pas *.polycorp.test : seul l'IdP doit recevoir des tickets.

3.4 Keycloak

Keycloak 26.8.1 tourne dans Docker Compose avec l'application (fichier docker-compose.yml du dépôt). J'ai monté un krb5.conf qui ne permet que les chiffrements AES et le keytab. Toute la configuration du realm est faite par un script kcadm (keycloak/configure-keycloak.sh), ce qui la rend reproductible. J'ai ensuite vérifié chaque écran dans la console d'administration.

Fédération AD. Fournisseur LDAP ad-polycorp, vendor Active Directory, connexion ldap://dc01.polycorp.test:389 avec le bind svc-kc-ldap, base OU=People,DC=polycorp,DC=test en One Level, nom d'utilisateur sAMAccountName, identifiant objectGUID, mode READ_ONLY. Des mappers ajoutent title, userPrincipalName et les groupes de OU=Groups.

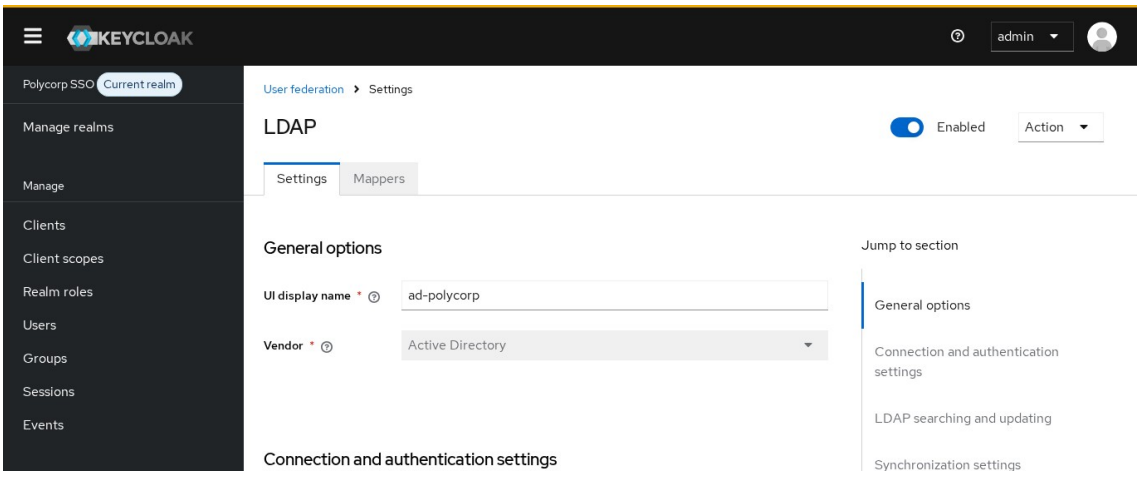


Figure 2 – Fédération LDAP : connexion à DC01, vendor Active Directory.

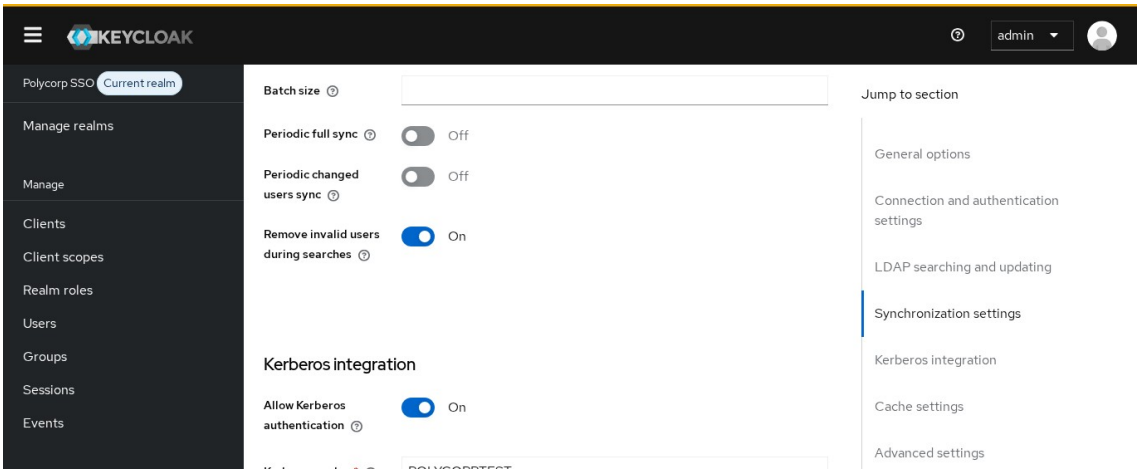


Figure 3 – Intégration Kerberos : realm POLYCORP.TEST, principal HTTP/sso.polycorp.test, keytab.

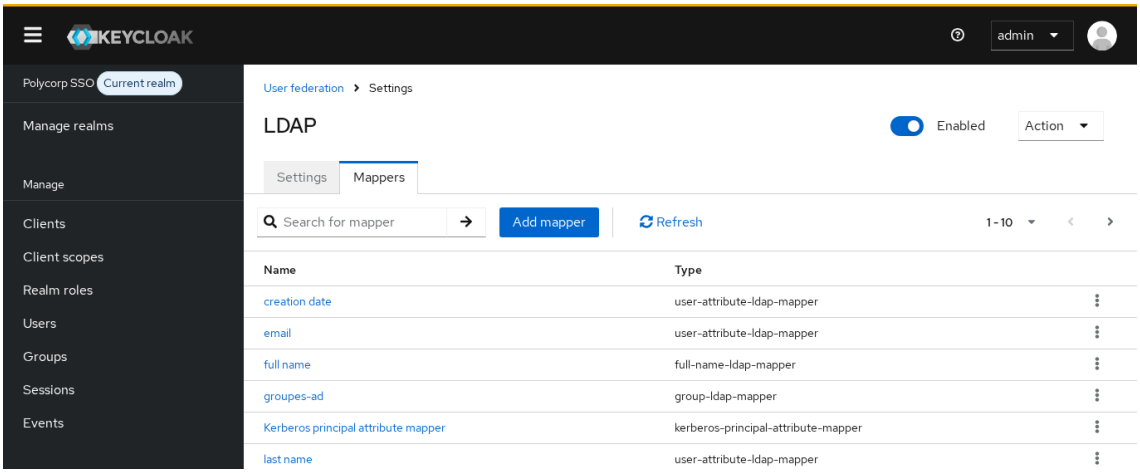


Figure 4 – Mappers LDAP : groupes, title, upn, Kerberos principal, MSAD account controls.

L'option « Use Kerberos for password authentication » est aussi activée. Si quelqu'un tape son mot de passe dans le formulaire de Keycloak, c'est le KDC qui le vérifie, et pas un bind LDAP. Keycloak ne stocke donc aucun mot de passe : dans la console, Bob a une seule « credential » de type password, fournie par ad-polycorp.

Synchronisation. Après une synchronisation complète, les cinq utilisateurs de People et les groupes sont importés, chacun avec son lien de fédération :

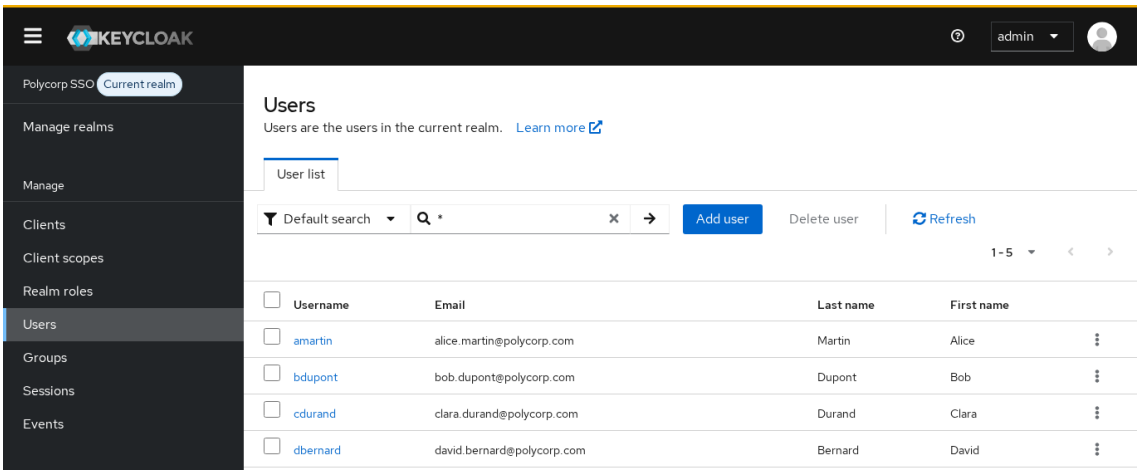


Figure 5 – Utilisateurs importés depuis AD.

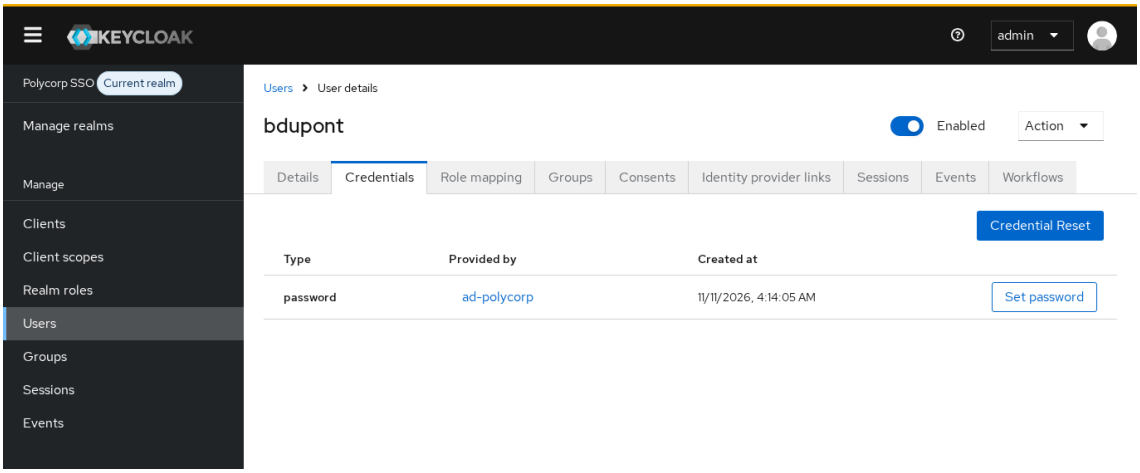


Figure 6 – Bob : mot de passe fourni par ad-polycorp, jamais stocké dans Keycloak.

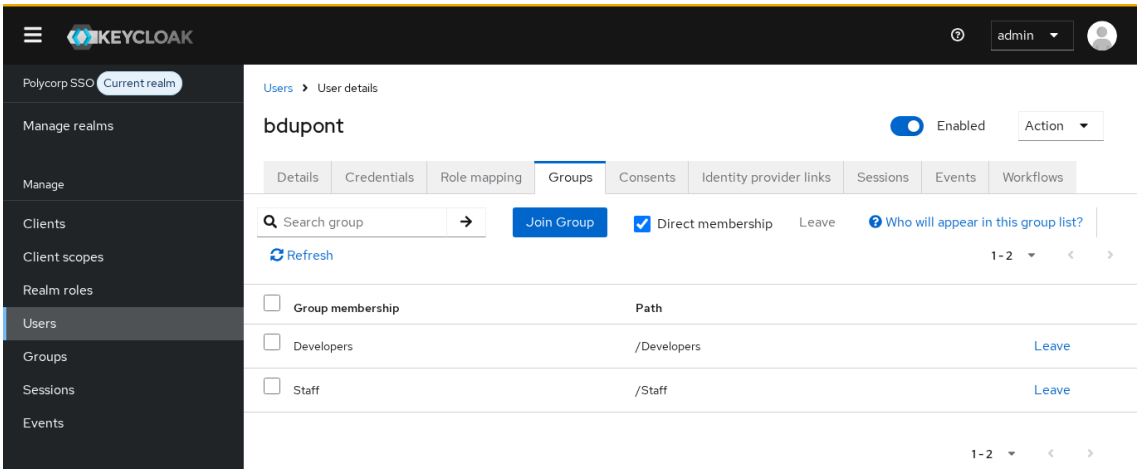


Figure 7 – Bob : groupes Developers et Staff, lus dans AD.

Flux de connexion. Dans le flux browser, j'ai passé l'étape Kerberos de Disabled à ALTERNATIVE. Elle est essayée en premier. Si le navigateur ne renvoie pas de ticket, le flux continue vers Cookie, puis vers le formulaire. Je l'ai laissée en ALTERNATIVE et pas en REQUIRED : en REQUIRED, un utilisateur hors domaine ne pourrait plus jamais se connecter.

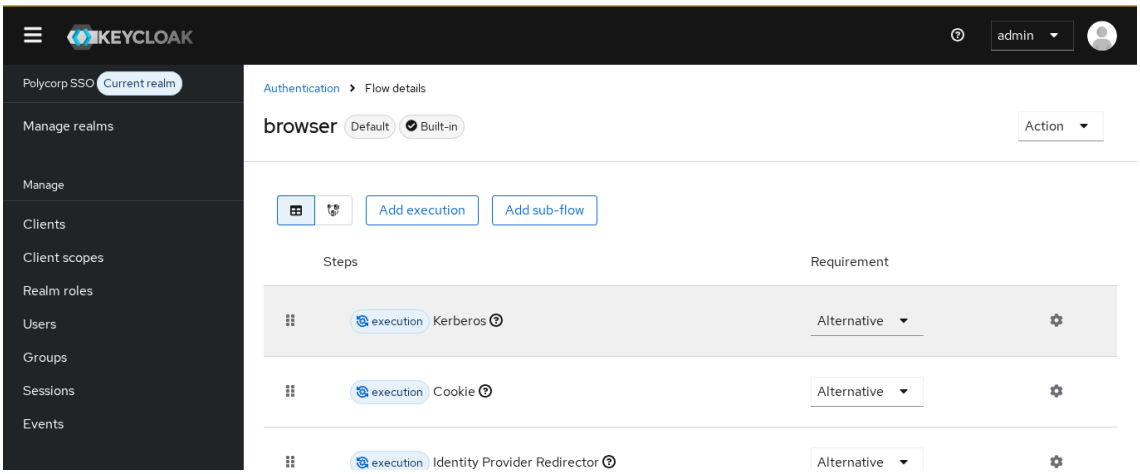


Figure 8 – Flux browser : Kerberos en Alternative, avant Cookie et le formulaire.

Client SAML. Client http://app.polycorp.test:8181/saml/metadata avec ACS http://app.polycorp.test:8181/saml/acs, documents et assertions signés (RSA-SHA256), NameID = nom d'utilisateur, et des mappers pour email, givenName, sn, upn, title et memberOf (groupes AD).

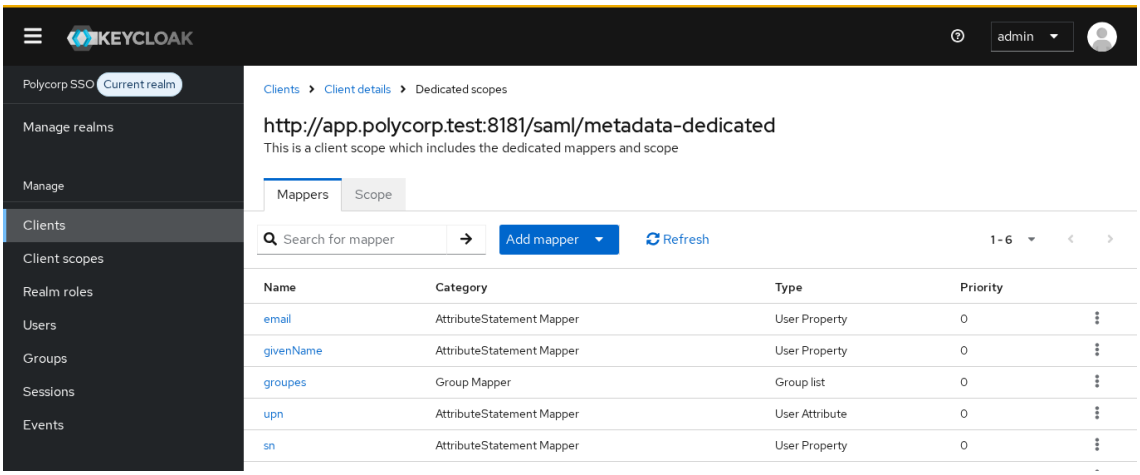


Figure 9 – Mappers du client SAML : attributs AD placés dans l'assertion.

Application. C'est le fournisseur de service SAML du TP4 (python3-saml), qui vérifie strictement la signature, l'Audience, le Recipient, InResponseTo, les dates et le rejeu. J'ai ajouté une seule chose : au démarrage, il charge tout seul les métadonnées de l'IdP depuis Keycloak (IDP_METADATA_URL), au lieu d'un fichier à importer à la main après chaque redémarrage.

4. Démonstration nominale : le SSO

4.1 L'application s'ouvre sans saisie

Bob ouvre sa session de domaine sur CL01, lance Edge et va sur http://app.polycorp.test:8181/rh. La page de l'application s'affiche directement, sans aucune page de connexion :

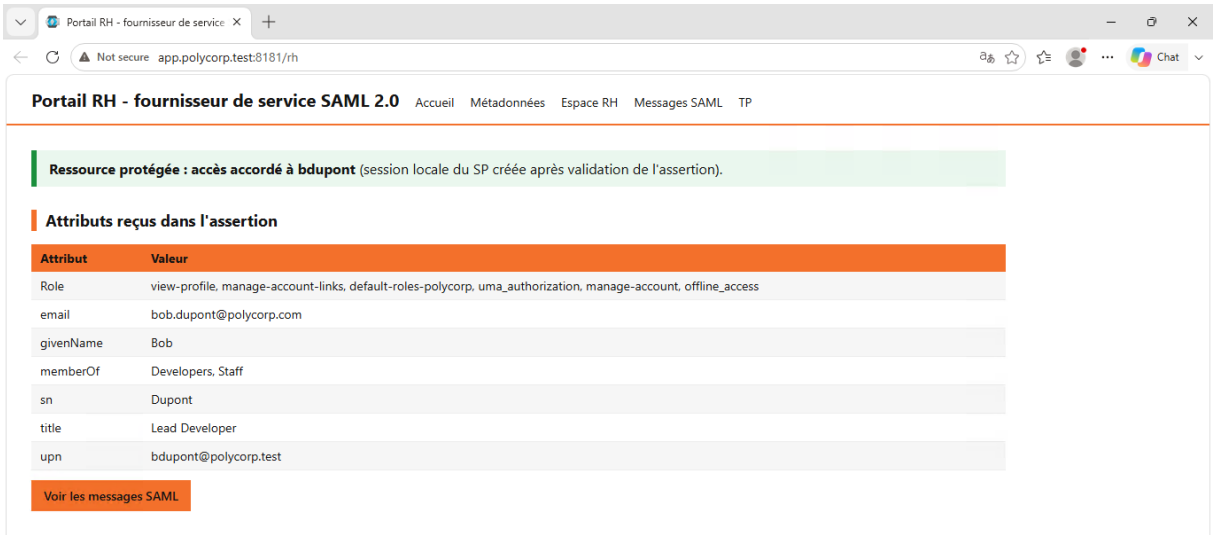


Figure 10 – Edge sur CL01 : accès accordé à bdupont, avec ses attributs AD, sans aucune saisie.

Dans la même session, klist montre le nouveau ticket de service :

```
#1>     Client: bdupont @ POLYCORP.TEST
        Server: HTTP/sso.polycorp.test @ POLYCORP.TEST
        KerbTicket Encryption Type: AES-256-CTS-HMAC-SHA1-96
        Ticket Flags 0x40a10000 -> forwardable renewable pre_authent name_canonicalize
        Start Time: 11/11/2026 4:21:31 (local)
        End Time:   11/11/2026 14:21:31 (local)
        Kdc Called: DC01.polycorp.test
```

Le ticket est pour HTTP/sso.polycorp.test, en AES-256, et il n'a pas le flag ok_as_delegate, contrairement à cifs/DC01 au TP3.5 : svc-kc-http n'est pas approuvé pour la délégation, donc Edge ne transmet pas de copie du TGT à Keycloak.

4.2 La trace sur DC01

```
04:21:31 4769 user=bdupont@POLYCORP.TEST service=svc-kc-http sid=S-1-5-21-…-1114
         ip=::ffff:192.168.50.20 port=52841 etype=0x12 status=0x0 options=0x40810000
```

L'événement 4769 a la même heure que le ticket (04:21:31). La demande vient de CL01 (192.168.50.20), et le Service Name est svc-kc-http : comme au TP3.5, le 4769 donne le compte qui porte le SPN, pas le SPN lui-même. Le chiffrement est 0x12 (AES-256) et le statut 0x0. Côté Keycloak, le journal SPNEGO montre la même seconde :

```
04:21:31,274 DEBUG [SPNEGOAuthenticator] SPNEGO Security context accepted with token length: 180,
             established: true, credDelegState: false, mutualAuthState: true
```

established: true veut dire que le ticket a été déchiffré avec le keytab et accepté. credDelegState: false confirme qu'aucun TGT n'a été délégué, et mutualAuthState: true que Keycloak s'est aussi authentifié auprès du navigateur.

4.3 L'assertion SAML et ses attributs AD

L'application affiche l'assertion reçue. En voici un extrait (attributs abrégés) :

```
<saml:Issuer>http://sso.polycorp.test:8080/realms/polycorp</saml:Issuer>
<saml:NameID Format="urn:oasis:names:tc:SAML:1.1:nameid-format:unspecified">bdupont</saml:NameID>
<saml:Audience>http://app.polycorp.test:8181/saml/metadata</saml:Audience>
<saml:AuthnStatement AuthnInstant="2026-11-11T04:21:31.321Z" SessionIndex="Ut6adGA-
NbrKATVdNdxDxJhF::a6fe…">
<saml:Attribute Name="email">      bob.dupont@polycorp.com
<saml:Attribute Name="givenName">  Bob
<saml:Attribute Name="sn">         Dupont
<saml:Attribute Name="upn">        bdupont@polycorp.test
<saml:Attribute Name="title">      Lead Developer
<saml:Attribute Name="memberOf">   Developers
<saml:Attribute Name="memberOf">   Staff
```

Toutes ces valeurs viennent d'AD : le mail et la fonction saisis au TP2, l'UPN, et les groupes Developers et Staff. L'AuthnInstant (04:21:31.321) est quelques millisecondes après l'acceptation du ticket. Cette assertion relie donc directement le ticket Kerberos de Bob à l'identité reçue par l'application.

4.4 Le dialogue HTTP Negotiate sur le réseau

Pour voir l'échange, j'ai capturé le trafic HTTP sur la VM sso (tcpdump) pendant que le script tests/verifier.ps1 refaisait le parcours depuis la session de Bob, après un klist purge :

```
 6  CL01 → app   GET /rh
 9  app  → CL01  302 → /connexion → 302 vers Keycloak (SAMLRequest)
15  CL01 → kc    GET /realms/polycorp/protocol/saml?SAMLRequest=…
16  kc   → CL01  HTTP/1.1 401 Unauthorized   WWW-Authenticate: Negotiate
21  CL01 → kc    GET … Authorization: Negotiate <SPNEGO negTokenInit : KRB5 AP-REQ, sname
HTTP/sso.polycorp.test, AES-256>
26  kc   → CL01  HTTP/1.1 200 OK           WWW-Authenticate: Negotiate oYGEMIGBoAMKAQCh…  (+
formulaire SAML auto-POST)
32  CL01 → app   POST /saml/acs  (SAMLResponse)
39  app  → CL01  200 OK  page protégée
```

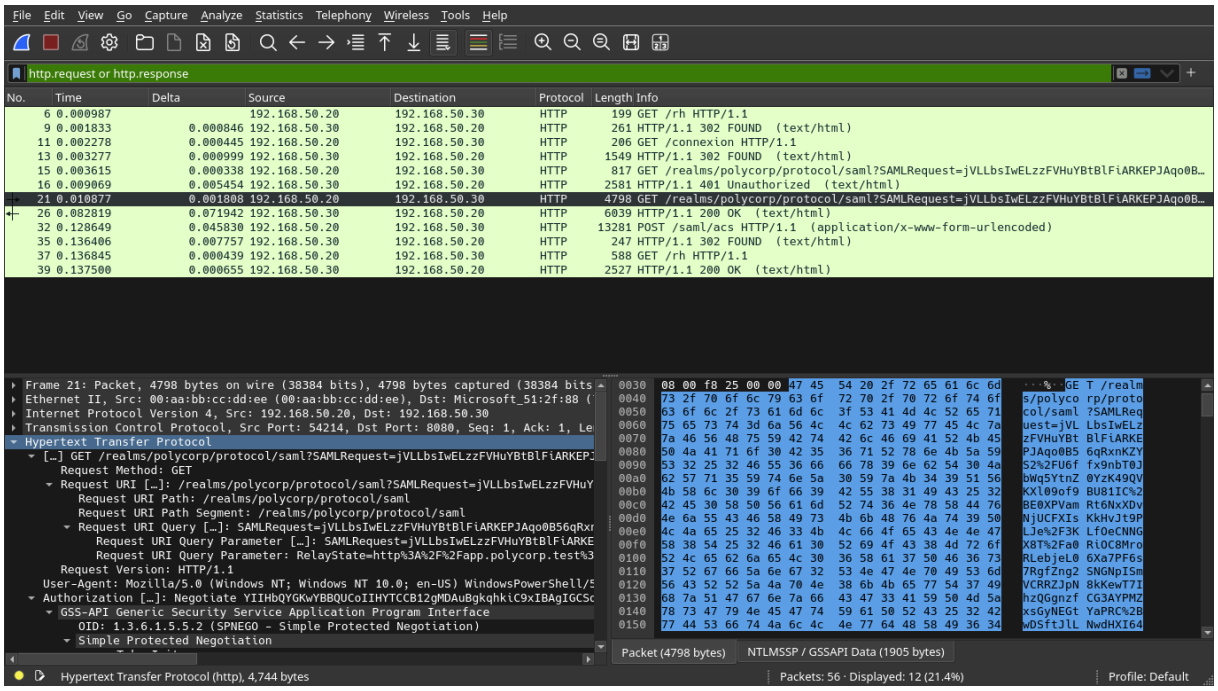


Figure 11 – Wireshark : trame 21, Authorization: Negotiate avec le jeton GSS-API / SPNEGO.

On retrouve exactement le mécanisme : un premier refus 401 qui demande Negotiate, puis la même requête avec le jeton SPNEGO. Ce jeton propose Kerberos (OID 1.2.840.113554.1.2.2), MS-Kerberos, NTLM et NEGOEX, et contient l'AP-REQ pour sname HTTP/sso.polycorp.test dans le realm POLYCORP.TEST. La réponse 200 contient elle aussi un jeton Negotiate : c'est l'AP-REP de l'authentification mutuelle. Le ticket est en clair dans la requête HTTP, puisque le labo n'a pas de TLS (voir partie 7), mais il est chiffré avec la clé du service et ne contient aucun mot de passe.

4.5 Script de vérification

Le dépôt contient tests/verifier.ps1, qui vérifie toute la chaîne depuis une session de domaine : DNS, SPN, ticket, GPO, puis SPNEGO et SAML. Il refait ce que fait le navigateur : il suit les redirections avec -UseDefaultCredentials, récupère la SAMLResponse renvoyée par Keycloak, puis la poste à l'ACS de l'application.

```
== 1. DNS
sso.polycorp.test -> 192.168.50.30
app.polycorp.test -> 192.168.50.30
== 2. SPN
CN=svc-kc-http,OU=Services,DC=polycorp,DC=test
HTTP/sso.polycorp.test
== 3. Ticket
A ticket to HTTP/sso.polycorp.test has been retrieved successfully.
KerbTicket Encryption Type: AES-256-CTS-HMAC-SHA1-96
Server: HTTP/sso.polycorp.test @ POLYCORP.TEST
== 4. GPO
AuthServerAllowlist = sso.polycorp.test
== 5. SPNEGO + SAML
Keycloak a accepte le ticket : SAMLResponse recue (13120 caracteres), destination http://app.polycorp.test:8181/saml/acs
OK : application ouverte pour bdupont sans saisie de mot de passe
PS C:\Windows\system32\WindowsPowerShell\v1.0>
```

Figure 12 – verifier.ps1 sur CL01 : tous les contrôles passent, l'application s'ouvre pour bdupont.

5. Démonstrations des refus

5.1 Hors domaine ou sans ticket : retour au formulaire

Poste hors domaine. Depuis mon PC Linux, qui n'est pas dans le domaine et n'a pas de ticket POLYCORP.TEST, la même adresse affiche la page de connexion de Keycloak. Le navigateur reçoit le 401 Negotiate mais n'a rien à envoyer, et le flux passe au formulaire.

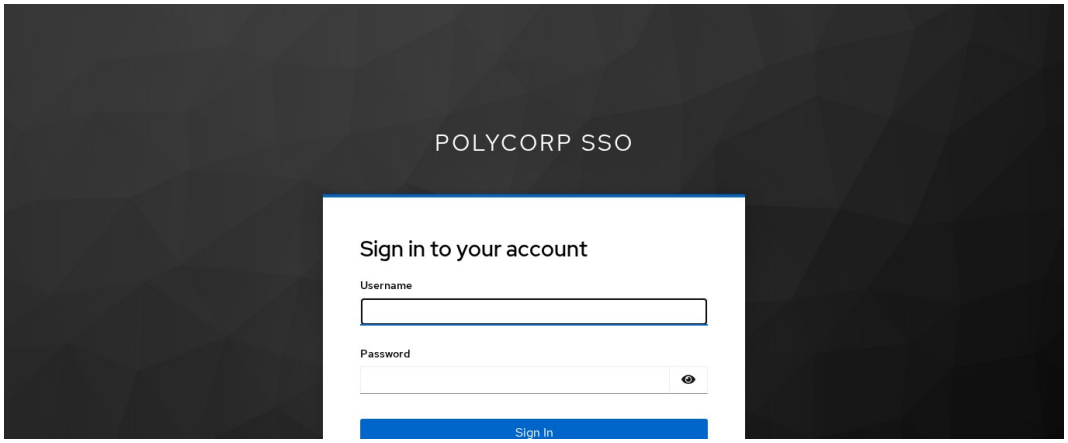


Figure 13 – Poste hors domaine (Linux) : pas de ticket, formulaire de connexion de Keycloak.

Session sans ticket sur CL01. J'ai ouvert une session avec le compte local CL01\labadmin. Elle n'a pas de TGT du domaine (klist vide). Le résultat est le même : formulaire de connexion.

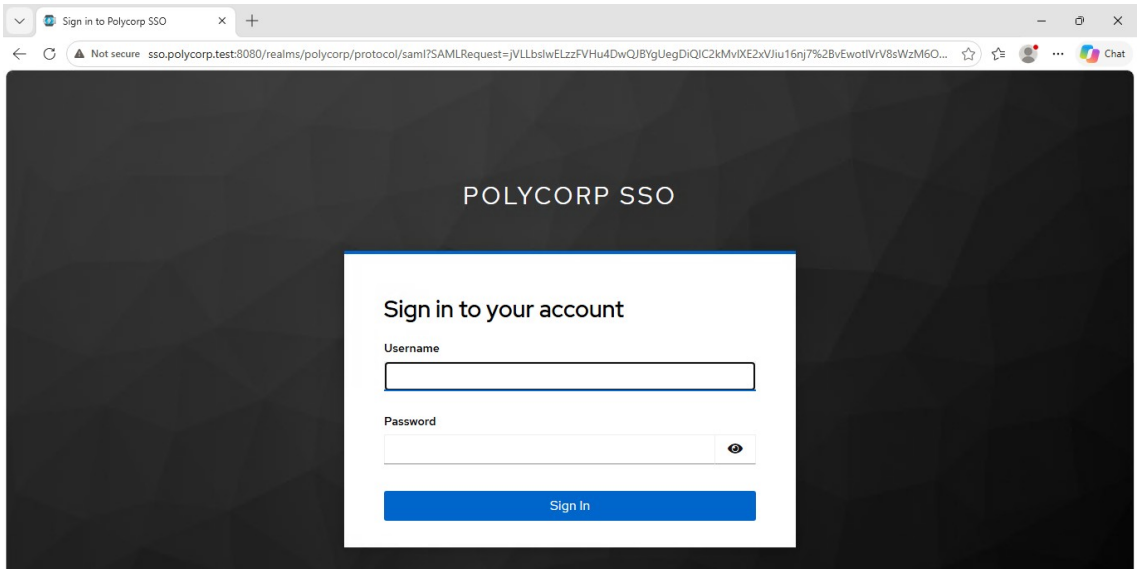


Figure 14 – Session locale labadmin sur CL01 : pas de ticket, Keycloak affiche son formulaire.

Dans ce formulaire, Bob peut quand même se connecter avec son mot de passe. Keycloak le vérifie auprès du KDC (« Principal bdupont@POLYCORP.TEST authenticated successfully » dans le journal), et l'application s'ouvre. Le SSO est un confort en plus, il ne bloque pas les autres cas.

5.2 Compte désactivé

J'ai désactivé Bob dans AD pendant que sa session Windows était ouverte, et fermé ses sessions Keycloak (sinon le cookie Keycloak l'aurait laissé entrer sans revérification, voir partie 6). Ensuite, dans la même session Windows :

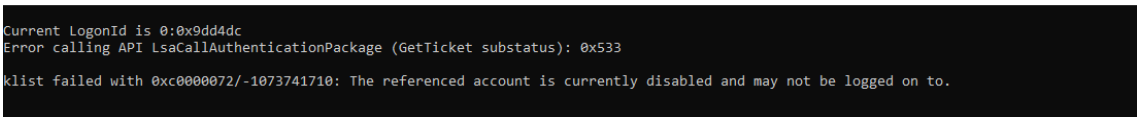


Figure 15 – klist get HTTP/sso.polycorp.test : « The referenced account is currently disabled ».

```
klist get HTTP/sso.polycorp.test
klist failed with 0xc0000072: The referenced account is currently disabled and may not be logged on
to.
== DC01, journal Security (audit des échecs activé)
04:32:28 4768 FAIL user=bdupont svc=krbtgt/POLYCORP.TEST status=0x12 ip=::ffff:192.168.50.20
port=53191
04:32:32 4768 FAIL user=bdupont svc=krbtgt/POLYCORP.TEST status=0x12 ip=::ffff:192.168.50.20
port=53192
```

Le KDC refuse de délivrer un ticket : le 4768 en échec a le code 0x12, KDC_ERR_CLIENT_REVOKED (« Clients credentials have been revoked »). Le ticket HTTP/sso n'existe donc pas. Dans Edge, l'authentification intégrée échoue, et comme le site est autorisé, Edge affiche la boîte « Sign in to access this site » de Windows :

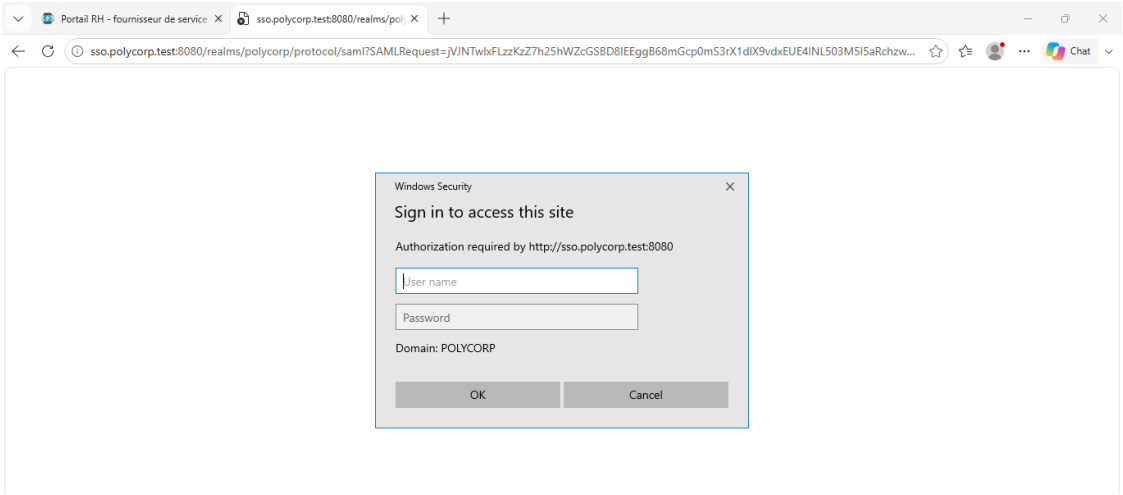


Figure 16 – Compte désactivé : pas de ticket, Edge demande des identifiants au lieu d'ouvrir l'application.

Après Cancel, Keycloak affiche son formulaire. J'y ai tapé le bon mot de passe de Bob, et Keycloak a aussi refusé, en passant par le KDC :

```
DEBUG [KerberosUsernamePasswordAuthenticator] Failed to authenticate user bdupont@POLYCORP.TEST:
      javax.security.auth.login.LoginException: Clients credentials have been revoked (18)
WARN  [org.keycloak.events] type="LOGIN_ERROR", realmName="polycorp",
      clientId="http://app.polycorp.test:8181/saml/metadata", ipAddress="192.168.50.20",
error="invalid_user_credentials"
```

Le code Kerberos 18 est le même KDC_ERR_CLIENT_REVOKED (0x12). La désactivation dans AD est donc appliquée tout de suite, par les deux chemins (SPNEGO et formulaire), sans aucune synchronisation, puisque la fédération est en lecture seule et sans cache. J'ai ensuite réactivé Bob.

Ce que montre ce test : la désactivation ne dépend pas de la durée du TGT. Même dans une session Windows déjà ouverte, le KDC vérifie l'état du compte quand on lui demande un nouveau ticket, et il refuse. Je n'ai vu sur DC01 que des 4768 en échec et aucun 4769, alors que j'avais aussi activé l'audit des échecs pour Kerberos Service Ticket Operations. Je pense que Windows, après le refus, retente un échange AS complet, mais je n'ai pas isolé ce point. À noter : par défaut, les échecs ne sont pas audités dans ces sous-catégories, j'ai dû les activer pour voir ces refus.

5.3 Décalage d'horloge

J'ai arrêté chrony sur la VM sso et avancé son horloge de 10 minutes (04:35:11 → 04:45:11), au-delà de la tolérance Kerberos de 5 minutes. CL01 et DC01 restent à l'heure. Bob obtient sans problème son ticket HTTP/sso, puisque le KDC est à l'heure, mais Keycloak le refuse :

```
WARN [SPNEGOAuthenticator] SPNEGO login failed: GSSException: Failure unspecified at GSS-API level
     (Mechanism level: Clock skew too great (37))
```

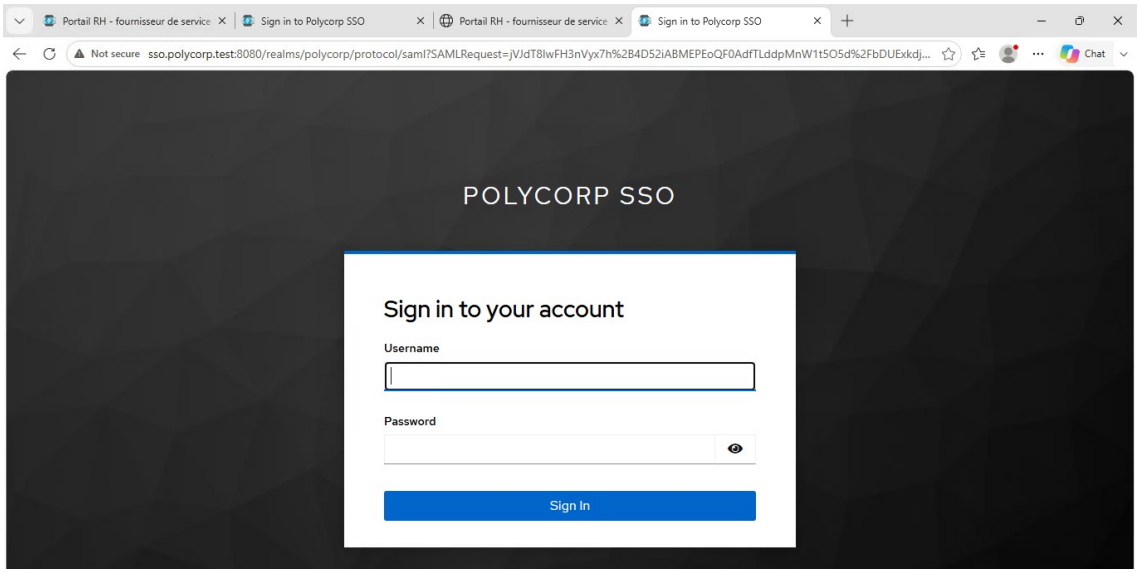


Figure 17 – Horloge de l'IdP décalée de 10 min : SPNEGO refusé, retour au formulaire.

L'erreur 37 est KRB_AP_ERR_SKEW. L'authenticator envoyé par CL01 est daté de 04:35, et pour Keycloak il est 04:45, donc l'écart est hors tolérance et l'AP-REQ est rejeté. C'est la protection contre le rejeu vue au TP3 : sans horloge commune, on ne peut pas savoir si un authenticator est récent. Ce cas est le symétrique de celui du TP3.5 : là, c'est le service qui est décalé et pas le client. Il n'y a donc pas de correction automatique, et rien n'apparaît sur DC01, puisque le KDC a bien fait son travail. Le refus n'est visible que dans les journaux de Keycloak, et l'utilisateur voit seulement le formulaire.

Remise en état : chrony relancé, chronyc makestep a corrigé l'horloge de 600 s, puis l'écart était de l'ordre de la microseconde. Ce retour en arrière de l'horloge a bloqué les conteneurs (Keycloak ne répondait plus et l'application ne chargeait plus les métadonnées de l'IdP). Un redémarrage des deux conteneurs a suffi, puis le SSO a remarché tout de suite. Ça montre qu'en production, l'heure doit être synchronisée par NTP sur le domaine, et qu'il ne faut jamais la corriger brutalement sur un serveur en marche.

6. Analyse de sécurité

Ce qui protège la chaîne.

- Le mot de passe ne quitte jamais le poste. Keycloak ne voit qu'un ticket de service chiffré pour lui, avec une durée limitée (10 h), lié à une clé de session et protégé contre le rejeu par l'horodatage.
- Le SPN est porté par un compte dédié avec une clé AES-256 aléatoire longue. Un attaquant du domaine peut demander un ticket pour ce SPN et essayer de casser la clé hors ligne (Kerberoasting) : c'est le risque principal d'un compte de service avec SPN. Le mot de passe long et aléatoire, et le refus de RC4, le rendent irréaliste à casser. Un gMSA (compte de service géré, mot de passe changé automatiquement) serait encore mieux, mais ktpass a besoin du mot de passe pour fabriquer le keytab, donc j'ai gardé un compte classique.
- Délégation interdite : le ticket présenté à Keycloak ne contient pas de TGT transféré (credDelegState: false). Si Keycloak était compromis, il ne pourrait pas se faire passer pour Bob auprès d'autres services.
- La GPO n'autorise que sso.polycorp.test. Sans elle, le navigateur n'envoie pas de ticket, et un site malveillant ne pourrait pas en demander un.

– Côté SAML : assertions signées, Audience et Recipient contrôlés, InResponseTo vérifié, durée courte et rejeu refusé, comme testé au TP4.

– AD reste la seule source de vérité : fédération READ_ONLY sans cache, compte de bind en lecture seule, désactivation prise en compte immédiatement.

Limites observées.

– La session Keycloak (cookie) et la session de l'application continuent après la désactivation du compte, jusqu'à leur expiration ou une déconnexion. Pendant mon test, il a fallu fermer les sessions Keycloak à la main (script kc_logout_user.sh). En production, il faudrait automatiser cette révocation, ou raccourcir les sessions.

– Le jeton Negotiate passe en HTTP clair. Il n'est pas réutilisable ailleurs (il est lié au service et à l'horodatage), mais la session qui en sort peut être volée. TLS est indispensable en production.

7. Différences avec un déploiement en production

Dans le labo	En production
HTTP sur 8080 et 8181	HTTPS partout (certificat de l'AC d'entreprise), Keycloak derrière un reverse proxy
LDAP 389 avec bind simple	LDAPS 636 ou StartTLS, ou bind Kerberos avec le même keytab
Keycloak start-dev, base H2, admin temporaire	start avec PostgreSQL, compte admin nominatif et MFA, au moins deux nœuds
Compte de service avec mot de passe	gMSA ou rotation planifiée du mot de passe et du keytab
Une seule VM pour l'IdP et l'application	Séparation IdP / applications, supervision des événements LOGIN_ERROR
chrony arrêté à la main pour le test	NTP sur toute la chaîne (DC, IdP, postes), alerte si l'écart dépasse 1 minute

8. Problèmes rencontrés et solutions

Problème	Cause	Solution
Taille de VM indisponible (SkuNotAvailable)	Capacité Azure en France Central	Taille équivalente B2as_v2 / B2als_v2
Plus d'accès RDP/SSH en cours de travail	Mon adresse IP publique a changé, le NSG la filtrait	Mise à jour des règles NSG
Le test du compte désactivé passait quand même	Session Keycloak déjà ouverte (cookie)	Fermeture des sessions de l'utilisateur avant le test
Fenêtre InPrivate : identifiants demandés même avec un compte actif	Edge désactive l'authentification intégrée en InPrivate	Fenêtre normale pour les tests SPNEGO
Refus Kerberos invisibles dans le journal	Audit des échecs désactivé par défaut	auditpol : Success and Failure
Keycloak bloqué après la correction d'horloge	Retour en arrière de 10 minutes	Redémarrage des conteneurs
verifier.ps1 : « formulaire » alors que le SSO marchait	Keycloak renvoie un formulaire SAML auto-POST, que PowerShell ne soumet pas	Le script poste lui-même la SAMLResponse à l'ACS

9. Dépôt du projet

Le dépôt projet-sso-kerberos-keycloak contient tout ce qu'il faut pour reconstruire le projet. Aucun secret n'est versionné : .env, keytab et mots de passe sont exclus par .gitignore, et seul .env.example est fourni.

```
docker-compose.yml           Keycloak 26.8.1 + application
keycloak/krb5.conf           realm POLYCORP.TEST, AES uniquement
keycloak/configure-keycloak.sh   kcadm : realm, federation AD, Kerberos, flux, client SAML
keycloak/kc_logout_user.sh   fermeture des sessions d'un utilisateur
app/                         SP SAML (TP4) + chargement automatique des metadonnees
infra/azure/01_create_lab.sh reseau, NSG et 3 VM
infra/ad/*.ps1               foret, OU/comptes/groupes, comptes de service + SPN + keytab, GPO
tests/verifier.ps1           verification de bout en bout depuis CL01
```

Conclusion

Le projet réunit tout le cours dans un seul parcours : l'annuaire et les groupes du TP1 et du TP2, Kerberos et les SPN du TP3 et du TP3.5, et SAML du TP4. Le point clé est que chaque brique fait confiance à la précédente avec une preuve cryptographique : le KDC au mot de passe, Keycloak au ticket (keytab), l'application à l'assertion (signature). Les démonstrations de refus montrent que cette confiance est contrôlée à chaque étape : sans ticket on revient au formulaire, un compte désactivé est refusé par le KDC, et une horloge fausse fait rejeter le ticket.

Si je continuais le projet, j'ajouterais TLS sur toute la chaîne, un gMSA pour le compte de service, et un client OIDC pour montrer que le même ticket Kerberos peut aussi donner un access token pour une API, comme au TP4.